

Unsupervised Learning

Unit 4: Clustering, k -means, Gaussian Mixtures & EM, Principal Component Analysis

Dr. Gopal Chandra Jana

Dept. of CSE, Sharda University
Machine Learning (CSE473) — Unit 4

July 23, 2026

Course Page:

<https://www.gcjana.in/courses/shardauniversity/2601/machine-learning/>

Outline

- 1 Unsupervised Learning and Clustering Methods
- 2 k -means, Gaussian Mixtures and EM
- 3 Principal Component Analysis
- 4 Summary and Practice

Learning Outcomes of Unit 4

After this unit, a student will be able to...

- 1 **Define** unsupervised learning and distinguish its task families.
- 2 **Compute** distance measures and cluster-quality criteria numerically.
- 3 **Apply** hierarchical clustering with single, complete and average linkage.
- 4 **Derive** the k -means objective and **execute** Lloyd's algorithm by hand.
- 5 **Formulate** Gaussian mixture models and **perform** one EM iteration.
- 6 **Derive** PCA from the maximum-variance principle and **compute** a full eigen-decomposition.
- 7 **Relate** PCA to the SVD and state its properties and limitations.

Mapping to the Syllabus

A. Unsupervised learning; clustering methods (distance-based, probabilistic, hierarchical). **B.** k -means, initialization, applications; GMM and EM. **C.** PCA: components, covariance, SVD, properties, limitations, applications.

What is Unsupervised Learning?

Definition

Given only the inputs

$$S = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}, \quad \mathbf{x}_i \in \mathbb{R}^n$$

with **no labels** y_i , discover structure in the data.

Formally

We seek to model the **density** $p(\mathbf{x})$ or find a compact **representation** of the data, rather than a mapping $\mathbf{x} \mapsto y$.

The core difficulty

There is **no ground truth** and therefore no unambiguous error measure. Success is judged by *usefulness*, internal validity indices, or downstream performance.

Why bother without labels?

- Labels are **expensive**; raw data is cheap and abundant.
- **Exploration**: what natural groups exist in my data?
- **Compression** and denoising.
- **Pre-processing** for a supervised model (features, dimension reduction).
- **Anomaly detection**: what does *not* fit the structure?

Task families

Clustering, **dimensionality reduction**, density estimation, association rules, representation learning.

Supervised vs. Unsupervised — Side by Side

Aspect	Supervised	Unsupervised
Data	$(\mathbf{x}_i, y_i)$ pairs	$\mathbf{x}_i$ only
Goal	learn $f : \mathcal{X} \rightarrow \mathcal{Y}$	discover structure in $p(\mathbf{x})$
Feedback	the correct answer	none
Evaluation	accuracy, RMSE on a test set	internal indices, expert judgement
Typical tasks	classification, regression	clustering, PCA, density estimation
Risk	overfitting the labels	finding structure that is not there

The middle ground

Semi-supervised (few labels + many unlabelled) and **self-supervised** (labels manufactured from the data itself, e.g. predict a masked word) — see Unit 1.

A warning

A clustering algorithm **always returns clusters** — even on pure noise. Always ask whether the structure found is *real*.

What is Clustering?

Definition

Partition S into groups $C_1, \dots, C_K$ such that

$$\bigcup_{k=1}^K C_k = S, \quad C_i \cap C_j = \emptyset \ (i \neq j)$$

for a **hard**, **exhaustive** partition.

The two goals

- **High intra-cluster similarity** — points in a cluster are close to each other (*cohesion*).
- **Low inter-cluster similarity** — different clusters are far apart (*separation*).

Variants

Soft/fuzzy clustering assigns degrees of membership; **hierarchical** clustering produces nested partitions.

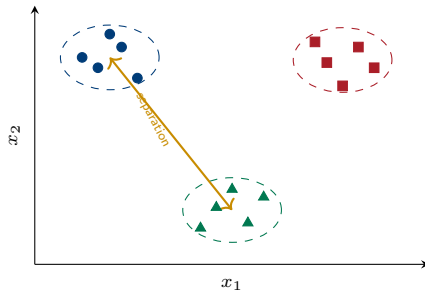


Figure: Compact, well-separated clusters — the ideal case.

Distance Measures — The Mathematics

Minkowski distance of order p (the general family)

$$d_p(\mathbf{x}, \mathbf{z}) = \left(\sum_{j=1}^n |x_j - z_j|^p \right)^{1/p}$$

Special cases

- $p = 1$: **Manhattan** (city-block) $\sum_j |x_j - z_j|$
- $p = 2$: **Euclidean** $\sqrt{\sum_j (x_j - z_j)^2}$
- $p \rightarrow \infty$: **Chebyshev** $\max_j |x_j - z_j|$

Other useful measures

Cosine: $\cos \theta = \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\| \|\mathbf{z}\|}$

Mahalanobis: $d = \sqrt{(\mathbf{x} - \mathbf{z})^\top \Sigma^{-1} (\mathbf{x} - \mathbf{z})}$

Metric axioms

A valid distance satisfies, for all $\mathbf{x}, \mathbf{y}, \mathbf{z}$:

- 1 $d(\mathbf{x}, \mathbf{y}) \geq 0$, $= 0$ iff $\mathbf{x} = \mathbf{y}$
- 2 $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (symmetry)
- 3 $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$ (triangle)

Choosing wisely

Euclidean for compact isotropic clusters; **cosine** for text/high dimensions (magnitude irrelevant); **Mahalanobis** when features are correlated or on different scales.

Always scale features first — otherwise the largest-range feature dominates the distance.

Numerical Example — Distance Measures

Take $\mathbf{x} = (2, 3)$ and $\mathbf{z} = (5, 7)$, so $|\Delta| = (3, 4)$

Measure	Computation	Value
Manhattan ($p = 1$)	$ 3 + 4 $	7
Euclidean ($p = 2$)	$\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25}$	5
Minkowski $p = 3$	$(3^3 + 4^3)^{1/3} = (27 + 64)^{1/3} = 91^{1/3}$	4.498
Chebyshev ($p \rightarrow \infty$)	$\max(3, 4)$	4
Cosine similarity	$\frac{2(5) + 3(7)}{\sqrt{13} \sqrt{74}} = \frac{31}{31.016}$	0.9995

Observe the ordering

$d_1 \geq d_2 \geq d_3 \geq \dots \geq d_\infty$: $7 > 5 > 4.498 > 4$.
Minkowski distance **decreases monotonically** as p increases.

Note the cosine value

$\cos \theta \approx 0.9995$ — the two vectors point in almost the same *direction* even though they are 5 units apart. **Direction and distance measure different things.**

Cluster Quality — Internal Criteria

Sum of squared errors (SSE / WCSS)

$$\text{SSE} = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2$$

Lower is better — but SSE **always decreases** as K grows, so it cannot by itself choose K .

Interpreting $s(i)$

- $s \approx 1$: well inside its cluster.
- $s \approx 0$: on the boundary between two clusters.
- $s < 0$: probably in the **wrong** cluster.

The mean $\bar{s}$ over all points scores the whole clustering.

Silhouette coefficient

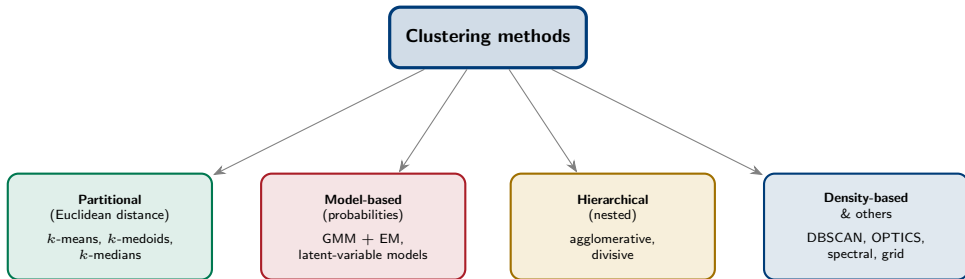
For a point i let $a(i)$ be its mean distance to points in its *own* cluster and $b(i)$ the mean distance to the nearest *other* cluster. Then

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$$

Other indices

Davies–Bouldin (lower better), **Calinski–Harabasz** (higher better), **Dunn index** (higher better). If true labels *are* available, use **ARI** or **NMI** (external validation).

Taxonomy of Clustering Methods



Choosing a family

Partitional: fast, needs K , assumes spherical clusters.

Model-based: soft assignment, elliptical clusters, principled likelihood.

Hierarchical: no K needed, gives a dendrogram, but $O(m^2)$ – $O(m^3)$.

Density-based: arbitrary shapes, finds outliers, no K needed.

Method Based on Euclidean Distance (Partitional)

The principle

Represent each cluster by a **prototype** and assign every point to its nearest prototype under a Euclidean (or other) metric:

$$C_k = \{\mathbf{x} : \|\mathbf{x} - \boldsymbol{\mu}_k\| \leq \|\mathbf{x} - \boldsymbol{\mu}_j\| \forall j\}$$

This partitions space into **Voronoi cells** — so the cluster boundaries are **linear** (perpendicular bisectors).

Family members

- **k-means**: prototype = mean; minimises squared error.
- **k-medoids** (PAM): prototype = an actual data point; robust to outliers.
- **k-medians**: prototype = median; minimises L_1 .

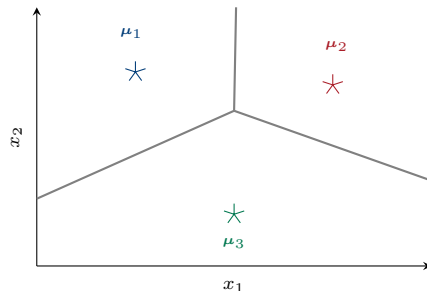


Figure: Prototypes induce a *Voronoi tessellation*; each cell is one cluster.

Method Based on Probabilities (Model-Based)

The principle

Assume the data is **generated** by a mixture of K probability distributions:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k p(\mathbf{x} \mid \theta_k), \quad \sum_k \pi_k = 1, \pi_k \geq 0$$

Clustering then becomes **parameter estimation**: find $\{\pi_k, \theta_k\}$ maximising the likelihood.

Advantages over distance-based methods

- **Soft** assignment — a point may belong 70% to one cluster.
- Clusters may be **elliptical** and of different sizes/orientations.
- Provides a **generative model**: we can sample new data.
- Principled model selection via **AIC/BIC**.

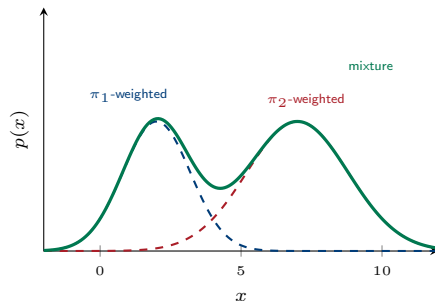


Figure: A mixture density is a weighted sum of component densities.

Hierarchical Clustering Methods

Two directions

- **Agglomerative** (bottom-up, AGNES): start with m singleton clusters; repeatedly **merge** the two closest until one remains.
- **Divisive** (top-down, DIANA): start with one cluster and repeatedly **split**. More costly ($2^{m-1} - 1$ possible splits).

Agglomerative algorithm

- 1 Compute the $m \times m$ distance matrix.
- 2 Merge the pair of clusters with the smallest linkage distance.
- 3 Update the matrix for the new cluster.
- 4 Repeat until a single cluster remains.

Complexity: $O(m^3)$ naively, $O(m^2 \log m)$ with heaps.

Linkage criteria — the mathematics

$$\text{Single: } d(A, B) = \min_{a \in A, b \in B} d(a, b)$$

$$\text{Complete: } d(A, B) = \max_{a \in A, b \in B} d(a, b)$$

$$\text{Average: } d(A, B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(a, b)$$

$$\text{Centroid: } d(A, B) = \|\mu_A - \mu_B\|$$

$$\text{Ward: } \Delta = \frac{|A||B|}{|A| + |B|} \|\mu_A - \mu_B\|^2$$

Behaviour

Single linkage can *chain* (elongated clusters); **complete** favours compact ones; **Ward** minimises the increase in SSE and is the common default.

Numerical Example — Hierarchical Clustering

Five points on a line: $P_1=2$, $P_2=5$, $P_3=9$, $P_4=10$, $P_5=15$

d	P_1	P_2	P_3	P_4	P_5
P_1	0	3	7	8	13
P_2	3	0	4	5	10
P_3	7	4	0	1	6
P_4	8	5	1	0	5
P_5	13	10	6	5	0

Single linkage — merge sequence

- 1 $\{P_3, P_4\}$ at $d = 1$ (smallest entry)
- 2 $\{P_1, P_2\}$ at $d = 3$
- 3 $\{P_1, P_2\} + \{P_3, P_4\}$ at $\min(7, 8, 4, 5) = 4$
- 4 $+ P_5$ at $\min(13, 10, 6, 5) = 5$

Complete linkage — a *different* tree

$\{P_3, P_4\}$ at 1; $\{P_1, P_2\}$ at 3; then P_5 joins $\{P_3, P_4\}$ at $\max(6, 5) = 6$; final merge at 13.

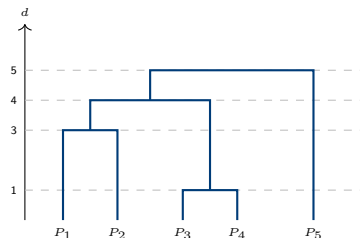


Figure: Single-linkage dendrogram. **Cutting** at $d = 4.5$ gives two

Quick Check — Round 1 (Clustering Basics)

[Q1]

Compute the Euclidean and Manhattan distances between $(1, 2)$ and $(4, 6)$.

[Q2]

A point has $a(i) = 2$ and $b(i) = 6$. Compute its silhouette coefficient and interpret it.

[Q3]

Using the distance matrix on the previous slide, which two clusters merge *third* under **average** linkage, and at what distance?

[Hint: after $\{P_3, P_4\}$ and $\{P_1, P_2\}$, compare $d(\{P_1 P_2\}, \{P_3 P_4\})$ and $d(P_5, \{P_3 P_4\})$.]

[Q4]

Why can SSE never be used on its own to choose the number of clusters K ?

Answers — Round 1

[A1]

$\Delta = (3, 4)$: Euclidean $= \sqrt{9 + 16} = 5$; Manhattan $= 3 + 4 = 7$.

[A2]

$s = \frac{b - a}{\max(a, b)} = \frac{6 - 2}{6} = 0.667$ — comfortably positive, so the point sits **well inside** its own cluster.

[A3]

$d(\{P_1 P_2\}, \{P_3 P_4\}) = \frac{7+8+4+5}{4} = 6.0$; $d(P_5, \{P_3 P_4\}) = \frac{6+5}{2} = 5.5$.

Since $5.5 < 6.0$, P_5 **merges with** $\{P_3, P_4\}$ at $d = 5.5$ — the *same* ordering as complete linkage, and different from single linkage.

[A4]

Because SSE **decreases monotonically** with K , reaching 0 when $K = m$ (every point its own cluster). We must look for an **elbow**, or use a criterion that penalises complexity (silhouette, BIC, gap statistic).

The k -means Objective Function

Formal statement

Introduce binary indicators $r_{ik} \in \{0, 1\}$ with $\sum_k r_{ik} = 1$ (point i belongs to exactly one cluster). Minimise the **distortion**

$$J(\{r_{ik}\}, \{\mu_k\}) = \sum_{i=1}^m \sum_{k=1}^K r_{ik} \|\mathbf{x}_i - \mu_k\|^2$$

Step 1 — optimise r with μ fixed

J is linear in r_{ik} and the terms are independent, so simply assign each point to its **nearest centroid**:

$$r_{ik} = \begin{cases} 1 & k = \arg \min_j \|\mathbf{x}_i - \mu_j\|^2 \\ 0 & \text{otherwise} \end{cases}$$

Step 2 — optimise μ with r fixed

J is quadratic in μ_k ; differentiate and set to zero:

$$\frac{\partial J}{\partial \mu_k} = -2 \sum_i r_{ik} (\mathbf{x}_i - \mu_k) = 0$$

$$\implies \boxed{\mu_k = \frac{\sum_i r_{ik} \mathbf{x}_i}{\sum_i r_{ik}}}$$

— the **mean** of the points assigned to cluster k . Hence the name.

Alternating these two exact minimisations is precisely **Lloyd's algorithm**.

The Standard Algorithm (Naive k -means / Lloyd)

Algorithm

- 1 **Initialise** K centroids $\mu_1, \dots, \mu_K$.
- 2 **Assignment step**: assign each x_i to the nearest centroid.
- 3 **Update step**: recompute each centroid as the mean of its members.
- 4 **Repeat** 2–3 until assignments stop changing (or J changes by $< \epsilon$).

Convergence guarantee

Each step **never increases** J , and there are finitely many possible assignments (K^m), so the algorithm **must terminate** in finitely many steps.

But only to a *local* optimum

J is non-convex; different initialisations give different results. Global k -means optimisation is **NP-hard** in general.

Remedy: run several times with different seeds and keep the lowest J .

Complexity

$$O(m K n t)$$

for m points, K clusters, n dimensions, t iterations — **linear in m** , which is why k -means scales so well.

Numerical Example — k -means, Iteration 1

Data ($m = 6$, $K = 2$), initial centroids $\mu_1 = A(1, 1)$, $\mu_2 = B(1.5, 2)$

$A(1, 1)$ $B(1.5, 2)$ $C(3, 4)$ $D(5, 7)$ $E(3.5, 5)$ $F(4.5, 5)$

Assignment step (Euclidean distance)

Point	$d(\cdot, \mu_1)$	$d(\cdot, \mu_2)$	Cluster
$A(1, 1)$	0.000	1.118	1
$B(1.5, 2)$	1.118	0.000	2
$C(3, 4)$	3.606	2.500	2
$D(5, 7)$	7.211	6.103	2
$E(3.5, 5)$	4.717	3.606	2
$F(4.5, 5)$	5.315	4.243	2

e.g. $d(C, \mu_1) = \sqrt{(3-1)^2 + (4-1)^2} = \sqrt{13} = 3.606$.

Update step

$\mu_1 = (1, 1)$ (only A)

$$\begin{aligned}\mu_2 &= \left(\frac{1.5+3+5+3.5+4.5}{5}, \frac{2+4+7+5+5}{5} \right) \\ &= \left(\frac{17.5}{5}, \frac{23}{5} \right) = (3.50, 4.60)\end{aligned}$$

Distortion after iteration 1

$$J = 0 + [(1.5-3.5)^2 + (2-4.6)^2] + \dots = 20.70$$

Numerical Example — k -means, Iterations 2–3

Iteration 2 — reassign with $\mu_1=(1, 1)$, $\mu_2=(3.5, 4.6)$

Point	$d(\cdot, \mu_1)$	$d(\cdot, \mu_2)$	Cluster
A	0.000	4.383	1
B	1.118	3.280	1 (moved)
C	3.606	0.781	2
D	7.211	2.830	2
E	4.717	0.400	2
F	5.315	1.077	2

New centroids: $\mu_1 = (1.25, 1.50)$, $\mu_2 = (4.00, 5.25)$; $J = 7.875$.

Iteration 3 — check for change

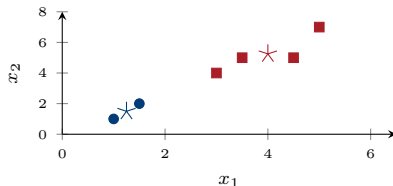
Re-assigning with the new centroids gives *exactly the same* clusters $\{A, B\}$ and $\{C, D, E, F\}$, so the centroids do not move.

⇒ **CONVERGED**

Distortion history

$J : 20.70 \rightarrow 7.875 \rightarrow 7.875$

Monotone decrease, then a fixed point — exactly as the theory predicts.



Initialization Methods

Classical choices

- **Forgy**: pick K **random data points** as centroids.
- **Random partition**: assign each point to a random cluster, then compute centroids (starts near the global mean).
- **Multiple restarts**: run R times, keep the lowest J (`n_init`).

k -means++ (Arthur & Vassilvitskii, 2007)

- 1 Choose the first centre uniformly at random.
- 2 For every point compute $D(\mathbf{x})$ = distance to the nearest chosen centre.
- 3 Choose the next centre with probability

$$P(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{x}'} D(\mathbf{x}')^2}$$

- 4 Repeat until K centres are chosen, then run Lloyd.

Why k -means++ works

It spreads the initial centres apart — points far from existing centres are **quadratically more likely** to be chosen. It guarantees

$$\mathbb{E}[J] \leq 8(\ln K + 2) J_{\text{OPT}}$$

i.e. $O(\log K)$ -competitive with the optimum, *before* Lloyd even starts.

The bad case it avoids

With plain random init, two centres can land in the *same* true cluster, splitting it while merging two others — a poor local optimum that Lloyd cannot escape.

k -means++ is the default in `scikit-learn`.

Numerical Example — k -means++ Initialization

Same six points; first centre chosen at random = $A(1, 1)$

Point	coords	$D(\mathbf{x}) = d(\mathbf{x}, A)$	$D(\mathbf{x})^2$	$P(\mathbf{x}) = D^2 / \sum D^2$
A	(1, 1)	0.000	0.000	0.0000
B	(1.5, 2)	1.118	1.250	0.0107
C	(3, 4)	3.606	13.000	0.1113
D	(5, 7)	7.211	52.000	0.4454
E	(3.5, 5)	4.717	22.250	0.1906
F	(4.5, 5)	5.315	28.250	0.2420
		Total	116.750	1.0000

Sample computation

$$D(D)^2 = (5 - 1)^2 + (7 - 1)^2 = 16 + 36 = 52, \text{ so}$$
$$P(D) = \frac{52}{116.75} = 0.4454.$$

Reading the result

D , the **farthest** point, is chosen with 44.5% probability, while the near-neighbour B has almost no chance (1.1%) — exactly the spreading behaviour we want.

Choosing K — Elbow and Silhouette

The elbow method

Plot J (WCSS) against K . J falls steeply while genuine clusters are being separated, then flattens. The **elbow** marks a good K .

Better criteria

- **Silhouette**: choose K maximising $\bar{s}$.
- **Gap statistic**: compare $\log J$ with that of uniform random reference data.
- **BIC/AIC**: for the model-based (GMM) view.

Reality

The elbow is often **ambiguous**. Combine numerical criteria with **domain knowledge** — how many segments does the business actually need?

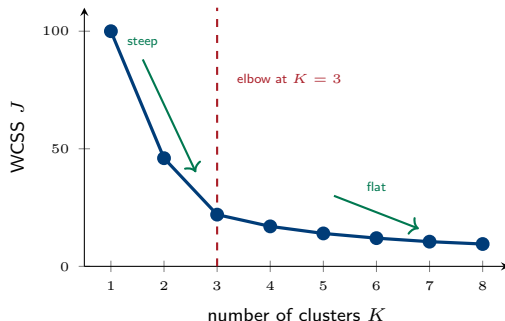


Figure: The elbow — adding a 4th cluster buys little extra reduction.

Limitations of k -means

Assumptions that can fail

- Clusters are **spherical** and of **similar size and density**.
- Boundaries are **linear** (Voronoi cells).
- K must be **specified in advance**.
- Sensitive to **outliers** (means are not robust).
- Sensitive to **initialization** (local optima).
- Requires **numeric** features and **scaling**.
- **Hard** assignment loses uncertainty information.

Remedies

- Non-spherical shapes $\Rightarrow$ **DBSCAN**, spectral clustering.
- Elliptical clusters, soft assignment $\Rightarrow$ **GMM** (next).
- Outliers $\Rightarrow$ **k -medoids**.
- Categorical data $\Rightarrow$ **k -modes**.
- Poor initialization $\Rightarrow$ **k -means++**, many restarts.

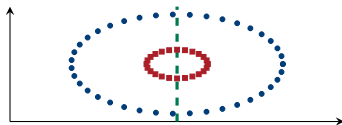


Figure: k -means cuts concentric rings with a *line* — and fails.

Application 1 — Vector Quantization

The idea

Replace every data vector by the **index** of its nearest centroid. The set of K centroids is the **codebook**; k -means is the standard way to build it.

$$\mathbf{x} \mapsto \arg \min_k \|\mathbf{x} - \boldsymbol{\mu}_k\| \mapsto \text{index } k$$

Reconstruction: $\hat{\mathbf{x}} = \boldsymbol{\mu}_k$ (lossy).

Bit accounting

Storing K codewords needs $\log_2 K$ bits per vector instead of the full representation.

Numeric: image colour quantization

A 512×512 image at 24-bit colour, quantized to $K = 256$ colours:

Original $512^2 \times 24$	6,291,456 bits = 768 KB
Codebook 256×24	6,144 bits
Indices $512^2 \times \log_2 256$	2,097,152 bits
Total	2,103,296 bits = 256.8 KB
Compression	2.99 : 1

Uses

image/speech compression, codebooks for retrieval, product quantization for approximate nearest-neighbour search.

Applications 2 and 3 — Cluster Analysis and Feature Learning

Cluster analysis (exploration)

- **Customer segmentation** — group by purchase behaviour, then design targeted offers.
- **Document/topic grouping** of news or research papers.
- **Image segmentation** — cluster pixels by colour/texture.
- **Bioinformatics** — gene-expression groups.
- **Anomaly detection** — points far from every centroid.

Feature learning

Cluster centroids can serve as a **new feature basis**. Replace $\mathbf{x}$ by the vector of its distances (or similarities) to the K centroids:

$$\phi(\mathbf{x}) = (\|\mathbf{x} - \boldsymbol{\mu}_1\|, \dots, \|\mathbf{x} - \boldsymbol{\mu}_K\|)$$

or by a **one-hot / triangle** encoding of cluster membership.

Why it helps

This is a cheap, **unsupervised** way to build non-linear features — Coates & Ng showed k -means features rival more complex methods for image classification. It is the same idea as choosing **RBF centres** (Unit 3).

Gaussian Mixture Models — The Model

Definition

A GMM models the density as a weighted sum of K Gaussians:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_{k=1}^K \pi_k = 1, \quad \pi_k \geq 0$$

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

Latent-variable view

Introduce $\mathbf{z} \in \{0, 1\}^K$, one-hot, with $p(z_k = 1) = \pi_k$.
Then

$$p(\mathbf{x} \mid z_k = 1) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

and marginalising $\mathbf{z}$ recovers the mixture. **The cluster label is the missing data.**

Responsibility (posterior)

By Bayes' rule,

$$\gamma(z_{ik}) \equiv p(z_k = 1 \mid \mathbf{x}_i) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

This is the **soft assignment** of point i to cluster k , with $\sum_k \gamma(z_{ik}) = 1$.

Why Maximum Likelihood is Hard Here

The log-likelihood

$$\ln p(\mathbf{X} \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{i=1}^m \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

The obstruction

The **sum sits inside the logarithm**, so the log does not decompose over components and setting the derivative to zero gives **no closed-form solution**.

A second problem — singularities

If one component collapses onto a single point ($\boldsymbol{\mu}_k = \mathbf{x}_i$, $\sigma_k \rightarrow 0$), the likelihood $\rightarrow \infty$. The ML problem is **ill-posed**; regularise the covariance or restart.

The insight behind EM

If we knew the responsibilities $\gamma(z_{ik})$, the parameter updates would be easy (weighted means and covariances). If we knew the parameters, the responsibilities would be easy (Bayes).

So alternate between the two — that is Expectation–Maximization.

Setting derivatives to zero gives

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i \gamma(z_{ik}) \mathbf{x}_i, \quad N_k = \sum_i \gamma(z_{ik})$$

— but γ itself depends on $\boldsymbol{\mu}_k$, so this is a **fixed-point** equation, not a solution.

The EM Algorithm for GMM

Algorithm

① **Initialise** μ_k, Σ_k, π_k (often from k -means) and evaluate the log-likelihood.

② **E-step** — compute responsibilities with the current parameters:

$$\gamma(z_{ik}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)}$$

③ **M-step** — re-estimate the parameters using those responsibilities:

$$N_k = \sum_{i=1}^m \gamma(z_{ik}), \quad \mu_k^{\text{new}} = \frac{1}{N_k} \sum_i \gamma(z_{ik}) \mathbf{x}_i, \quad \pi_k^{\text{new}} = \frac{N_k}{m}$$

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_i \gamma(z_{ik}) (\mathbf{x}_i - \mu_k^{\text{new}})(\mathbf{x}_i - \mu_k^{\text{new}})^\top$$

④ **Check convergence** of the log-likelihood; otherwise return to step 2.

Guarantee: each EM iteration *never decreases* the log-likelihood.

Numerical Example — EM E-step

1-D data $X = \{1, 2, 4, 6, 7\}$; initial $\mu_1=2$, $\mu_2=6$, $\sigma_1^2=\sigma_2^2=4$, $\pi_1=\pi_2=0.5$

With $\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$ and $\sqrt{2\pi(4)} = 5.0133$.

Responsibilities

x	$\pi_1 \mathcal{N}_1$	$\pi_2 \mathcal{N}_2$	γ_1	γ_2
1	0.08802	0.00438	0.9526	0.0474
2	0.09974	0.01350	0.8808	0.1192
4	0.06049	0.06049	0.5000	0.5000
6	0.01350	0.09974	0.1192	0.8808
7	0.00438	0.08802	0.0474	0.9526

Sample computation for $x = 1$

$$\mathcal{N}_1 = \frac{1}{5.0133} e^{-(1-2)^2/8} = 0.17604$$

$$\mathcal{N}_2 = \frac{1}{5.0133} e^{-(1-6)^2/8} = 0.00877$$

$$\gamma_1 = \frac{0.5(0.17604)}{0.5(0.17604) + 0.5(0.00877)} = 0.9526$$

The teaching point

$x = 4$ is **exactly midway**, so $\gamma_1 = \gamma_2 = 0.5$ — perfect ambiguity. k -means would be forced to pick one; **GMM reports the uncertainty**.

Numerical Example — EM M-step

Component 1 (using the γ_1 column)

$$\begin{aligned} N_1 &= 0.9526 + 0.8808 + 0.5000 + 0.1192 + 0.0474 = \mathbf{2.5000} \\ \mu_1^{\text{new}} &= \frac{0.9526(1) + 0.8808(2) + 0.5(4) + 0.1192(6) + 0.0474(7)}{2.5} = \frac{5.7613}{2.5} = \mathbf{2.3045} \\ \pi_1^{\text{new}} &= \frac{N_1}{m} = \frac{2.5}{5} = \mathbf{0.500}, \quad (\sigma_1^2)^{\text{new}} = \frac{1}{N_1} \sum_i \gamma_{i1} (x_i - \mu_1^{\text{new}})^2 = \mathbf{2.3254} \end{aligned}$$

By symmetry, component 2

$$N_2 = 2.5, \quad \mu_2^{\text{new}} = \mathbf{5.6955}, \quad (\sigma_2^2)^{\text{new}} = \mathbf{2.3254}$$

What happened

The means moved **inward** ($2 \rightarrow 2.30$, $6 \rightarrow 5.70$) because the ambiguous point $x = 4$ pulls both, and the variances **shrank** ($4 \rightarrow 2.33$) as the model tightens around the data. The log-likelihood rose from $\mathbf{-11.232}$.

EM Properties and GMM vs. k -means

Properties of EM

- **Monotone**: ℓ never decreases.
- Converges to a **local** maximum (or saddle) — not necessarily global.
- **Slower** than k -means per iteration ($O(mKn^2)$ with full covariances).
- Sensitive to initialization $\Rightarrow$ commonly seeded by k -means.
- Applies far beyond GMM: missing data, HMMs, factor analysis.

	k -means	GMM + EM
Assignment	hard (0/1)	soft ($\gamma \in [0, 1]$)
Cluster shape	spherical	elliptical (any Σ)
Cluster size	implicitly equal	modelled by π_k
Objective	distortion J	log-likelihood ℓ
Output	centroids	full density model
Speed	fast	slower
Params per cluster	n	$n + \frac{n(n+1)}{2} + 1$

The relationship

k -means is the **limiting case** of EM for a GMM with $\Sigma_k = \varepsilon \mathbf{I}$ as $\varepsilon \rightarrow 0$: responsibilities become hard 0/1 and the M-step reduces to the plain mean.

Quick Check — Round 2 (k -means, GMM, EM)

[Q1]

A cluster contains $(2, 4)$, $(4, 6)$ and $(6, 8)$. Compute its centroid and its contribution to the SSE.

[Q2]

In k -means++ the first centre is at $(0, 0)$ and the candidates have $D^2 = \{4, 16, 20\}$. What is the selection probability of the second candidate?

[Q3]

Write the responsibility formula $\gamma(z_{ik})$ and state what it must sum to over k .

[Q4]

Under what limiting condition does EM for a GMM reduce to k -means?

Answers — Round 2

[A1]

Centroid = $(\frac{2+4+6}{3}, \frac{4+6+8}{3}) = (4, 6)$.

SSE = $[(2-4)^2 + (4-6)^2] + 0 + [(6-4)^2 + (8-6)^2] = 8 + 0 + 8 = 16$.

[A2]

$\sum D^2 = 4 + 16 + 20 = 40$, so $P = \frac{16}{40} = 0.40$ (40%).

[A3]

$\gamma(z_{ik}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$, and $\sum_{k=1}^K \gamma(z_{ik}) = 1$ for every point i — it is a posterior distribution over clusters.

[A4]

When all covariances are $\boldsymbol{\Sigma}_k = \varepsilon \mathbf{I}$ with $\varepsilon \rightarrow 0$: the responsibilities become hard 0/1 assignments to the nearest centre, and the M-step becomes the ordinary mean.

PCA — Motivation and Intuition

The goal

Find a **low-dimensional linear subspace** that retains as much of the data's variance as possible:

$$\mathbb{R}^n \rightarrow \mathbb{R}^d, \quad d \ll n$$

Two equivalent formulations

- 1 **Maximum variance**: choose directions along which the projected data varies most.
- 2 **Minimum reconstruction error**: choose the subspace minimising the mean squared distance from the points to their projections.

Pearson (1901) used the second; Hotelling (1933) the first. **They give the same answer.**

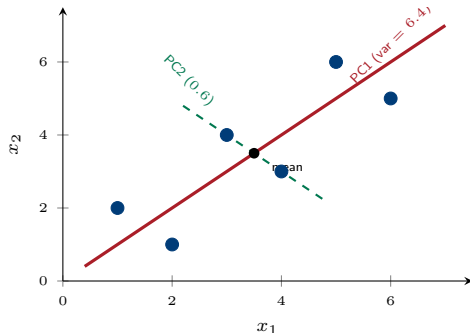


Figure: The data used in the full worked example below. PC1 captures the dominant spread.

The First Principal Component — Derivation

Set-up

Centre the data ($\bar{\mathbf{x}} = \mathbf{0}$). For a unit vector $\mathbf{u}_1$ the projection of $\mathbf{x}_i$ is $\mathbf{u}_1^\top \mathbf{x}_i$, and the **variance of the projections** is

$$\frac{1}{m-1} \sum_{i=1}^m (\mathbf{u}_1^\top \mathbf{x}_i)^2 = \mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1, \quad \mathbf{S} = \frac{1}{m-1} \sum_i \mathbf{x}_i \mathbf{x}_i^\top$$

Constrained maximisation

Maximise $\mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1$ subject to $\mathbf{u}_1^\top \mathbf{u}_1 = 1$ (otherwise we could scale $\mathbf{u}_1$ without limit). Lagrangian:

$$L = \mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1 + \lambda_1 (1 - \mathbf{u}_1^\top \mathbf{u}_1)$$

$$\frac{\partial L}{\partial \mathbf{u}_1} = 2\mathbf{S} \mathbf{u}_1 - 2\lambda_1 \mathbf{u}_1 = 0 \implies \boxed{\mathbf{S} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1}$$

Interpretation

$\mathbf{u}_1$ must be an **eigenvector** of the covariance matrix. Left-multiplying by $\mathbf{u}_1^\top$:

$$\mathbf{u}_1^\top \mathbf{S} \mathbf{u}_1 = \lambda_1$$

so the **variance captured equals the eigenvalue**. To maximise it, pick the eigenvector with the **largest** eigenvalue.

Further components

$\mathbf{u}_2$ maximises the variance subject to $\|\mathbf{u}_2\| = 1$ and $\mathbf{u}_2^\top \mathbf{u}_1 = 0$; this yields the eigenvector of the **second largest** eigenvalue, and so on. Since $\mathbf{S}$ is symmetric, the $\mathbf{u}_k$ are **orthogonal**.

Covariances and the PCA Algorithm

The covariance matrix

$$\mathbf{S} = \frac{1}{m-1} \mathbf{X}_c^\top \mathbf{X}_c, \quad S_{jk} = \frac{1}{m-1} \sum_{i=1}^m (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Symmetric, positive semi-definite, $n \times n$; the diagonal holds the variances.

Two key identities

$$\sum_{k=1}^n \lambda_k = \text{tr}(\mathbf{S}) = \text{total variance}$$

$$\text{variance explained by PC}_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

PCA by the covariance method — the recipe

- 1 **Centre** the data: $\mathbf{x}_i \leftarrow \mathbf{x}_i - \bar{\mathbf{x}}$ (and **scale** if units differ $\Rightarrow$ correlation matrix).
- 2 Compute the **covariance matrix** $\mathbf{S}$.
- 3 Compute its **eigenvalues and eigenvectors**.
- 4 **Sort** eigenvalues in decreasing order.
- 5 Keep the top d eigenvectors as columns of $\mathbf{U}_d$.
- 6 **Project**: $\mathbf{Z} = \mathbf{X}_c \mathbf{U}_d$.

Reminder

Centring is essential; without it the first component merely points at the mean.

Numerical Example — PCA Steps 1 & 2

Data ($m = 6, n = 2$)

(1, 2), (2, 1), (3, 4), (4, 3), (5, 6), (6, 5)

Step 1 — centre the data

$$\bar{x}_1 = \frac{1+2+3+4+5+6}{6} = 3.5, \quad \bar{x}_2 = 3.5.$$

$x_1 - \bar{x}_1$	-2.5	-1.5	-0.5	0.5	1.5	2.5
$x_2 - \bar{x}_2$	-1.5	-2.5	0.5	-0.5	2.5	1.5

Step 2 — covariance matrix

$$s_{11} = \frac{6.25+2.25+0.25+0.25+2.25+6.25}{5} = \frac{17.5}{5} = 3.5$$

$$s_{22} = \frac{2.25+6.25+0.25+0.25+6.25+2.25}{5} = 3.5$$

$$s_{12} = \frac{3.75+3.75-0.25-0.25+3.75+3.75}{5} = \frac{14.5}{5} = 2.9$$

$$\mathbf{S} = \begin{pmatrix} 3.5 & 2.9 \\ 2.9 & 3.5 \end{pmatrix}$$

Numerical Example — PCA Step 3: Eigenvalues

Solve the characteristic equation $\det(\mathbf{S} - \lambda\mathbf{I}) = 0$

$$\begin{vmatrix} 3.5 - \lambda & 2.9 \\ 2.9 & 3.5 - \lambda \end{vmatrix} = (3.5 - \lambda)^2 - 2.9^2 = 0$$

$$(3.5 - \lambda)^2 = 8.41 \implies 3.5 - \lambda = \pm 2.9$$

$$\lambda_1 = 3.5 + 2.9 = 6.4, \quad \lambda_2 = 3.5 - 2.9 = 0.6$$

Verification via trace and determinant

$$\lambda_1 + \lambda_2 = 6.4 + 0.6 = 7.0 = \text{tr}(\mathbf{S}) \checkmark$$

$$\lambda_1 \lambda_2 = 6.4(0.6) = 3.84 = \det(\mathbf{S}) \checkmark$$

(since $\det = 3.5^2 - 2.9^2 = 12.25 - 8.41 = 3.84$).

Variance explained

$$\text{PC1} = \frac{6.4}{7.0} = 91.43\%$$

$$\text{PC2} = \frac{0.6}{7.0} = 8.57\%$$

Keeping **one** component retains over 91% of the total variance — a 50% reduction in dimension for under 9% information loss.

Numerical Example — PCA Step 4: Eigenvectors

Solve $(\mathbf{S} - \lambda\mathbf{I})\mathbf{u} = \mathbf{0}$

For $\lambda_1 = 6.4$:

$$\begin{pmatrix} 3.5 - 6.4 & 2.9 \\ 2.9 & 3.5 - 6.4 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} -2.9 & 2.9 \\ 2.9 & -2.9 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \mathbf{0} \implies -2.9u_1 + 2.9u_2 = 0 \Rightarrow u_1 = u_2$$

Normalise

Take $(1, 1)$ and divide by its norm $\sqrt{2}$:

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = (0.7071, 0.7071)^T$$

Similarly, for $\lambda_2 = 0.6$ we get $u_1 = -u_2$:

$$\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = (0.7071, -0.7071)^T$$

Checks

$$\mathbf{u}_1^T \mathbf{u}_2 = 0.5 - 0.5 = 0 \checkmark \text{ (orthogonal)}$$

$$\|\mathbf{u}_1\| = \sqrt{0.5 + 0.5} = 1 \checkmark \text{ (unit norm)}$$

Interpretation

PC1 points along the 45° line $x_2 = x_1$ — the direction in which the two features **increase together**, which is exactly the strong positive correlation $s_{12} = 2.9$ we computed.

Numerical Example — PCA Step 5: Projection

Project each centred point: $z_1 = \mathbf{u}_1^\top \mathbf{x}_c$, $z_2 = \mathbf{u}_2^\top \mathbf{x}_c$

Original	Centred	$z_1 = \frac{a+b}{\sqrt{2}}$	$z_2 = \frac{a-b}{\sqrt{2}}$
(1, 2)	(-2.5, -1.5)	-2.8284	-0.7071
(2, 1)	(-1.5, -2.5)	-2.8284	+0.7071
(3, 4)	(-0.5, +0.5)	0.0000	-0.7071
(4, 3)	(+0.5, -0.5)	0.0000	+0.7071
(5, 6)	(+1.5, +2.5)	+2.8284	-0.7071
(6, 5)	(+2.5, +1.5)	+2.8284	+0.7071

Verify the variances

$$\text{Var}(z_1) = \frac{2(2.8284)^2 \cdot 2 + 0}{5} = \frac{32}{5} = 6.4 = \lambda_1 \checkmark$$

$$\text{Var}(z_2) = \frac{6(0.7071)^2}{5} = \frac{3}{5} = 0.6 = \lambda_2 \checkmark$$

And the decorrelation

$$\text{Cov}(z_1, z_2) = 0 \checkmark$$

The principal components are **uncorrelated** by construction — PCA has *diagonalised* the covariance matrix.

Numerical Example — Reconstruction Error

Keep only PC1: $\hat{\mathbf{x}} = \bar{\mathbf{x}} + z_1 \mathbf{u}_1$

Original	z_1	Reconstruction $\hat{\mathbf{x}}$	Squared error
(1, 2)	-2.8284	(1.5, 1.5)	0.25 + 0.25 = 0.5
(2, 1)	-2.8284	(1.5, 1.5)	0.5
(3, 4)	0.0000	(3.5, 3.5)	0.5
(4, 3)	0.0000	(3.5, 3.5)	0.5
(5, 6)	+2.8284	(5.5, 5.5)	0.5
(6, 5)	+2.8284	(5.5, 5.5)	0.5
Total			3.0

The theorem, verified

The reconstruction error equals the variance in the discarded directions:

$$\text{Error} = (m - 1) \sum_{k=d+1}^n \lambda_k = 5(0.6) = 3.0 \checkmark$$

Why this matters

This is exactly why the **maximum-variance** and **minimum-error** formulations coincide: total variance is fixed, so maximising what we *keep* is the same as minimising what we *lose*.

Choosing the Number of Components

Criteria

- **Cumulative variance:** smallest d with

$$\frac{\sum_{k \leq d} \lambda_k}{\sum_j \lambda_j} \geq 0.90 \text{ or } 0.95$$

- **Scree plot:** look for the elbow.
- **Kaiser criterion:** keep $\lambda_k > 1$ (on the correlation matrix).
- **Cross-validation** on the downstream task.

Our example

$d = 1$ already gives 91.43% — comfortably above a 90% threshold, so we keep **one** component.

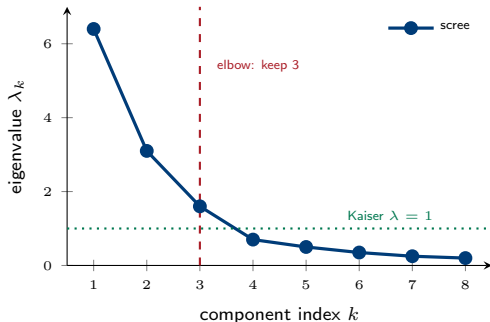


Figure: A scree plot for a larger problem — the elbow and the Kaiser line often broadly agree.

PCA via the Singular Value Decomposition

The SVD of the centred data matrix

Any $\mathbf{X}_c \in \mathbb{R}^{m \times n}$ factorises as

$$\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

with $\mathbf{U}$ ($m \times m$) and $\mathbf{V}$ ($n \times n$) orthogonal and $\mathbf{\Sigma}$ diagonal, holding the **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

The link to PCA

$$\mathbf{S} = \frac{\mathbf{X}_c^\top \mathbf{X}_c}{m-1} = \frac{\mathbf{V}\mathbf{\Sigma}^\top \mathbf{\Sigma} \mathbf{V}^\top}{m-1}$$

so the columns of $\mathbf{V}$ are the **principal directions**, and

$$\lambda_k = \frac{\sigma_k^2}{m-1}$$

Numeric check on our data

$\mathbf{X}_c^\top \mathbf{X}_c = \begin{pmatrix} 17.5 & 14.5 \\ 14.5 & 17.5 \end{pmatrix}$ has eigenvalues 32 and 3, so

$$\sigma_1 = \sqrt{32} = 5.657, \quad \sigma_2 = \sqrt{3} = 1.732$$

$$\lambda_1 = \frac{32}{5} = 6.4, \quad \lambda_2 = \frac{3}{5} = 0.6 \quad \checkmark$$

Why practitioners prefer the SVD

It avoids forming $\mathbf{X}_c^\top \mathbf{X}_c$, which **squares the condition number** and loses precision; it also handles $n \gg m$ well.

Properties of PCA

Mathematical properties

- 1 Components are **orthogonal** and the projections are **uncorrelated** — PCA diagonalises $\mathbf{S}$.
- 2 PC1 captures the **maximum possible** variance of any single direction; PC_k the maximum subject to orthogonality.
- 3 **Optimal linear reconstruction**: for any d , no other rank- d linear projection has lower squared error (Eckart–Young).
- 4 **Total variance is preserved**: $\sum_k \lambda_k = \text{tr}(\mathbf{S})$.
- 5 It is an **orthogonal rotation** of the coordinate system.

Practical benefits

- Removes **multicollinearity** — ideal before linear regression.
- **Denoises**: small- λ directions are often noise.
- Enables **2-D/3-D visualisation**.
- Reduces storage and training time.
- **Unsupervised** — needs no labels.

Related transforms

Whitening rescales each PC to unit variance ($z_k/\sqrt{\lambda_k}$); **Kernel PCA** applies the kernel trick for non-linear manifolds.

Limitations of PCA

Limitations

- 1 **Linear only** — cannot capture curved manifolds (use kernel PCA, t-SNE, UMAP, autoencoders).
- 2 **Variance $\neq$ importance**: a low-variance direction may be the one that separates the classes.
- 3 **Scale sensitive** — must standardise when units differ.
- 4 Components are **hard to interpret** (mixtures of all original features).
- 5 Assumes the data is well summarised by **mean and covariance** (implicitly Gaussian).
- 6 Sensitive to **outliers** (variance is not robust).
- 7 **Unsupervised** — ignores y entirely.

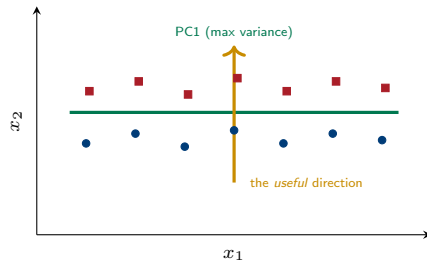


Figure: PC1 runs along the *horizontal* spread, but the class information lies *vertically* — keeping only PC1 would destroy it. (Use **LDA** when labels are available.)

Typical Applications of PCA

Computer vision

- **Eigenfaces** — face recognition by projecting images onto the top eigenvectors of the face covariance matrix. A 100×100 image (10,000 dims) is often well represented by ~ 100 components.
- Image compression and denoising.

Finance and engineering

- **Yield-curve analysis**: three components (level, slope, curvature) typically explain $> 95\%$ of bond-yield movements.
- **Risk factor** models for portfolios.
- **Process monitoring**: track T^2 statistics in the PC space.

Data analysis & science

- **Visualisation** of high-dimensional data in 2-D.
- **Genomics**: population structure from SNP data.
- **Chemometrics**: spectral data with thousands of wavelengths.

As a preprocessing step

PCA before regression (**principal component regression**) cures multicollinearity; PCA before k -means or an SVM speeds training and often improves accuracy by removing noise directions.

Quick Check — Round 3 (PCA)

[Q1]

For $\mathbf{S} = \begin{pmatrix} 5 & 2 \\ 2 & 5 \end{pmatrix}$, find both eigenvalues and the percentage of variance explained by PC1.

[Q2]

Find the unit eigenvector of $\mathbf{S}$ above corresponding to the larger eigenvalue.

[Q3]

The eigenvalues of a 4×4 covariance matrix are $\{8.0, 3.0, 0.8, 0.2\}$. How many components are needed to retain at least 90% of the variance?

[Q4]

A dataset has singular values $\sigma_1 = 6$ and $\sigma_2 = 2$ with $m = 10$ samples. What are λ_1 and λ_2 ?

Answers — Round 3

[A1]

$(5 - \lambda)^2 = 4 \Rightarrow 5 - \lambda = \pm 2$, so $\lambda_1 = 7$, $\lambda_2 = 3$ (check: sum = 10 = trace).

PC1 explains $\frac{7}{10} = 70\%$.

[A2]

$(\mathbf{S} - 7\mathbf{I})\mathbf{u} = \begin{pmatrix} -2 & 2 \\ 2 & -2 \end{pmatrix} \mathbf{u} = 0 \Rightarrow u_1 = u_2$, so $\mathbf{u}_1 = \frac{1}{\sqrt{2}}(1, 1)^\top = (0.707, 0.707)^\top$.

[A3]

Total = 12. Cumulative: $\frac{8}{12} = 66.7\%$; $\frac{11}{12} = 91.7\% \geq 90\%$. Hence **2 components** suffice.

[A4]

$\lambda_k = \frac{\sigma_k^2}{m-1}$: $\lambda_1 = \frac{36}{9} = 4.0$, $\lambda_2 = \frac{4}{9} = 0.444$.

Summary of Unit 4

What we covered

- ➊ **Unsupervised learning** — structure from unlabelled data; no ground truth.
- ➋ **Distances** — Minkowski family, cosine, Mahalanobis; metric axioms.
- ➌ **Cluster quality** — SSE/WCSS and the silhouette coefficient.
- ➍ **Method families** — distance-based (Voronoi), probabilistic (mixtures), hierarchical (dendrograms), density-based.
- ➎ **Hierarchical** — single, complete, average, centroid and Ward linkage; worked dendrogram.
- ➏ **k -means** — distortion J , the two exact minimisation steps, Lloyd's algorithm, convergence, $O(mKnt)$; worked to convergence.
- ➐ **Initialization** — Forgy, random partition, k -means++ with its D^2 rule and $O(\log K)$ guarantee.
- ➑ **Applications** — vector quantization (2.99:1), cluster analysis, feature learning.
- ➒ **GMM & EM** — mixture density, responsibilities, E- and M-steps, monotone likelihood; k -means as the $\varepsilon \rightarrow 0$ limit.
- ➓ **PCA** — Lagrangian derivation, $\mathbf{Su} = \lambda \mathbf{u}$, eigen-decomposition, projection, reconstruction error $(m-1) \sum_{k>d} \lambda_k$, SVD link $\lambda = \sigma^2/(m-1)$, properties, limitations, applications.

Practice Questions (Exam Oriented)

Short answer (2–5 marks)

- 1 Define unsupervised learning and give three task families.
- 2 Write the Minkowski distance and derive Manhattan, Euclidean and Chebyshev from it.
- 3 Define the silhouette coefficient and interpret $s = -0.3$.
- 4 State the k -means objective and derive the optimal centroid update.
- 5 Explain the k -means++ selection rule and why $D(\mathbf{x})^2$ is used.
- 6 Write the responsibility formula for a GMM and explain soft assignment.
- 7 State two properties and two limitations of PCA.

Long answer (8–10 marks) — include numerics

- 8 Given 6 points and 2 initial centroids, run k -means to convergence showing all distances, assignments, centroids and J .
- 9 Given a 5×5 distance matrix, build single- and complete-linkage dendrograms and compare them.
- 10 Derive the E- and M-step equations for a GMM; perform one full EM iteration on given 1-D data.
- 11 Derive the first principal component using a Lagrange multiplier and show that the variance captured equals λ_1 .
- 12 For a given dataset, compute the covariance matrix, eigenvalues, eigenvectors, projections, variance explained and reconstruction error.
- 13 Explain the relation between PCA and SVD, and verify it numerically.

Think & Explore (Take-Home)

Numeric drill

- Points $(2, 2), (3, 3), (8, 8), (9, 9)$ with $K = 2$: run k -means by hand and report J at each step.
- For $S = \begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix}$ compute both eigenvalues, eigenvectors and the variance explained.
- Build a complete-linkage dendrogram for five points you choose.

Reading

- Bishop, *PRML* — Ch. 9 (mixtures & EM), Ch. 12 (PCA).
- Hastie, Tibshirani & Friedman, *ESL* — Ch. 14.
- Han, Kamber & Pei, *Data Mining* — Ch. 10–11 (clustering).
- Jolliffe, *Principal Component Analysis*, Springer.

Coding (self-learning)

With `scikit-learn`: cluster a dataset with `KMeans` and plot the elbow and silhouette curves; fit a `GaussianMixture` and colour points by their responsibilities; run PCA and plot the cumulative explained-variance curve. Compare PCA with t-SNE on the same data.

Reflection

Cluster a dataset you know well. Do the discovered groups match your intuition? If not — is the algorithm wrong, the distance measure wrong, or your intuition wrong? **That question is the heart of unsupervised learning.**

Textbooks

- ① C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. [Ch. 9, 12]
- ② T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer. [Ch. 14]
- ③ J. Han, M. Kamber, J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed., Morgan Kaufmann.
- ④ I. T. Jolliffe, *Principal Component Analysis*, 2nd ed., Springer.
- ⑤ G. James et al., *An Introduction to Statistical Learning*, Springer.

Classic papers

- ⑥ S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, 28(2):129–137, 1982.
- ⑦ D. Arthur, S. Vassilvitskii, "*k*-means++: the advantages of careful seeding," *SODA*, 2007.
- ⑧ A. P. Dempster, N. M. Laird, D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *JRSS-B*, 39(1):1–38, 1977.
- ⑨ K. Pearson, "On lines and planes of closest fit to systems of points in space," *Phil. Mag.*, 2(11):559–572, 1901.

Thank You

Any Questions?

“Clustering is in the eye of the beholder.”

— Estivill-Castro, on the many valid ways to group the same data

Next: **Unit 5** — model evaluation, selection and advanced topics.

Post your doubts on the course page →

<https://www.gcjana.in/courses/shardauniversity/2601/machine-learning/#Post-doubts>